SIMULATING RAID LEVELS FOR FAULT TOLERANCE AND PERFORMANCE ANALYSIS

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

CSA0402 - OPERATING SYSTEM

to the award of the degree of

BACHELOR OF ENGINEERING

IN

AIML

Submitted by

M. HIMANATH SAI (192525246)

Under the Supervision of

Dr. Priskilla Angel Rani J

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

AUGUST 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "Simulating RAID Levels for Fault Tolerance and Performance Analysis" has been carried out by M. Himanath Sai (192525246) under the supervision of Priskilla Angel Rani J and is submitted in partial fulfilment of the requirements for the current semester of the AIML at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR	COURSE FACULTY
Dr. Anusaya S, Department of CSE, SIMATS Engineering, SIMATS, Chennai - 602105.	Priskilla Angel Rani J, Department of CSE, SIMATS Engineering, SIMATS, Chennai - 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

I, M. Himanath Sai (192525246) of the Department of AIML , SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "Simulating RAID Levels for Fault Tolerance and Performance Analysis" is the result of my own bonafide efforts. To the best of my knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date:

Signature of the Students with Names

M. Himanath Sai (192525246)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

Student Name / Reg. No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
B. Hemanjali (192525457)	RAID 0 & RAID 1 modules, disk model	Disk Manager class, striping and mirroring placement logic	Manual testing of striping and mirror-failure scenarios	Introduction, RAID Architecture (Ch.1, Ch.3-4.2)	20%
K. Vinay (192524278)	RAID 5 & RAID 6 parity engine	Parity calculation and distributed-parity placement logic	Single and dual disk-failure recovery testing	Literature Review, System Design (Ch.2, Ch.3)	20%
P. Harshavardhan (192524106)	RAID 10 module, Fault Injector	Mirror+stripe mapping, fault-injection state machine	Mixed mirror-pair failure testing	Implementation, Test Plan (Ch.4.1-4.3)	20%
N. Sathwik (192525077)	Recovery Manager, rebuild simulation	Rebuild/reconstruction routines for mirror & parity RAID	Rebuild verification and block-reconstruction testing	Results & Analysis (Ch.4.4-4.6)	20%
M. Himanath Sai (192525246)	Metrics Engine, workload generator, reporting	Metrics counters, comparative report generation	Performance-indicator verification across RAID levels	Conclusion, Appendices (Ch.5, Appendices)	20%
Total					100%

ABSTRACT

RAID (Redundant Array of Independent Disks) is an important storage technology used to organize multiple physical disk drives as a single logical storage system, improving storage performance, data availability, fault tolerance, and reliability. Instead of depending completely on a single disk, RAID distributes or duplicates information across several disks using techniques such as striping, mirroring, and parity, and each RAID level provides a different balance between usable capacity, performance, redundancy, and recovery capability.

This capstone project, "Simulating RAID Levels for Fault Tolerance and Performance Analysis," presents a software-based simulator that models common RAID configurations without requiring physical disk-array hardware. The simulator represents virtual disks, logical data blocks, data placement, disk health, read and write operations, disk failures, and recovery procedures for RAID 0, RAID 1, RAID 5, RAID 6, and RAID 10, allowing each level's behaviour to be observed and compared under a common simulation environment.

The core of the project applies modular Object-Oriented design to represent disks, RAID mapping policies, and recovery logic as reusable classes. RAID 0 uses striping across disks for parallelism but provides no redundancy; RAID 1 uses mirroring to maintain duplicate copies; RAID 5 uses distributed parity to tolerate a single disk failure; RAID 6 uses dual parity to tolerate two disk failures; and RAID 10 combines mirroring and striping for a balance of performance and redundancy. A dedicated Fault Injector marks selected virtual disks as failed, and the simulator evaluates whether the logical dataset remains available and whether missing data can be reconstructed by the Recovery Manager.

By implementing and testing these modules, the project demonstrates how disciplined data-placement design, fault injection, and reconstruction logic combine to produce an educational tool that makes RAID trade-offs directly observable rather than purely theoretical. The results show that RAID selection cannot be based on performance alone and must weigh capacity efficiency against fault tolerance for the target workload; the project provides a practical reference architecture for future storage-simulation and systems-education tools.

Keywords

RAID, Fault Tolerance, Disk Striping, Mirroring, Distributed Parity, Storage Simulation, Fault Injection, Data Recovery.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
RAID	Redundant Array of Independent Disks	-
I/O	Input / Output	-
FR	Functional Requirement	-
NFR	Non-Functional Requirement	-
CLI	Command-Line Interface	-
OS	Operating System	-
SSD	Solid State Drive	-
NVMe	Non-Volatile Memory Express	-
ms	Milliseconds	ms
%	Percentage	%

CHAPTER 1
INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Modern computing systems generate large volumes of data that must be stored reliably and accessed efficiently. Applications such as databases, cloud platforms, enterprise information systems, scientific computing, video processing, and online services depend on storage systems that can continue operating even when individual components fail. Since disk drives are mechanical or electronic devices with finite lifetimes, storage architecture must consider the possibility of disk failure. RAID was introduced as a method of combining multiple disks into a logical storage system, using multiple devices in a coordinated manner so that data can be distributed, duplicated, or protected using parity.

2 Need for the Project 1.

Learning RAID concepts has traditionally relied on static diagrams and theoretical descriptions of striping, mirroring, and parity. Diagrams alone do not demonstrate dynamic events such as a live disk failure, degraded-mode operation, or a rebuild in progress, and physical disk arrays are expensive and impractical for classroom use. A configurable, software-based RAID simulator is needed so that students and system designers can observe how each RAID level behaves under normal operation and under fault conditions, without requiring physical storage hardware.

1.3 Problem Statement

A single disk failure can cause data loss when no redundancy is present, while adding redundancy introduces capacity overhead and may affect write performance. Selecting a RAID level is therefore a multi-dimensional decision involving capacity, performance, and fault tolerance. The engineering problem addressed by this project is to design and implement a common simulation environment in which RAID 0, RAID 1, RAID 5, RAID 6, and RAID 10 can be configured, exercised with read/write workloads, subjected to injected disk failures, and evaluated for data availability and recoverability, so that their trade-offs become directly observable.

1.4 Project Aim

To design, implement, and test a software-based RAID simulator that models virtual disks, data placement, disk failures, and recovery for RAID 0, RAID 1, RAID 5, RAID 6, and RAID 10, and to comparatively analyse their fault tolerance and performance characteristics.

1.5 Project Objectives

- To study the architecture and working principles of common RAID levels.
- To develop a simulation model representing multiple disks and logical data blocks.
- To simulate normal read and write operations across RAID configurations.
- To inject disk failures and observe data availability under each RAID level.

- To compare storage capacity, fault tolerance, and performance indicators.
- To understand and simulate rebuild behaviour after disk replacement.
- To identify suitable RAID levels for different workload requirements.

1.6 Scope

Included: educational simulation of RAID 0, RAID 1, RAID 5, RAID 6, and RAID 10, covering logical disk modelling, block placement, redundancy, fault injection, rebuild simulation, and comparative performance/fault-tolerance indicators. Excluded: physical disk-array hardware, controller firmware, cache-algorithm modelling, and proprietary enterprise storage features. Assumptions: workloads are repeatable and simulated rather than measured from live hardware; a Python runtime environment is available. Boundary conditions: the simulator reports comparative simulation indicators rather than absolute hardware latency values.

1.7 Expected Engineering Outcome

The expected outcome is a working software prototype - a modular RAID simulator supporting RAID 0, RAID 1, RAID 5, RAID 6, and RAID 10 - together with a documented, tested simulation architecture covering fault injection and rebuild logic, and a comparative report that can serve as a reference for future storage-systems education tools.

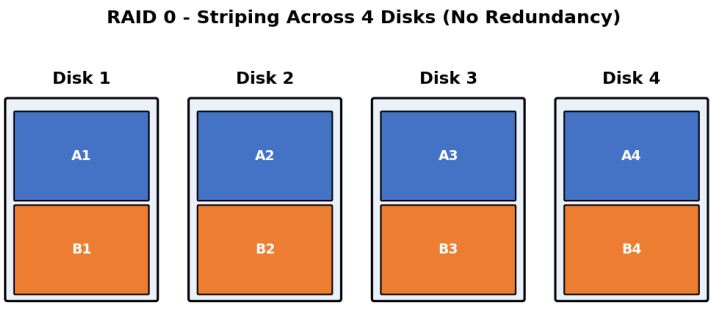

Figure 1: RAID 0 - Striping Architecture Across Four Disks

CHAPTER 2
LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature and established practice were identified through core operating-systems textbooks, the original RAID research paper, and widely cited references on storage systems, disk arrays, and fault-tolerant design. Sources were selected for direct relevance to the techniques used in this project: striping, mirroring, parity-based redundancy, and rebuild/reconstruction procedures.

2.2 Review of Existing Work

Storage systems provide persistent data retention for operating systems, applications, databases, and user information. Traditional architectures depend on individual disk drives, while modern architectures use arrays, distributed storage, solid-state devices, and cloud-based systems; a major design challenge is achieving reliability without making the system excessively expensive or slow. RAID emerged from research into combining inexpensive disks to obtain performance and reliability characteristics that were difficult to achieve with a single large disk, and RAID levels were subsequently standardised around different combinations of striping, mirroring, and parity so that designers can select a configuration according to workload and reliability needs.

Mirroring stores duplicate copies of data and therefore provides straightforward recovery when one copy is lost. Striping distributes blocks across disks and can increase parallelism. Parity-based systems calculate additional information that can be used to reconstruct missing data, and hybrid approaches combine these methods to balance speed and redundancy.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Single-disk storage (no RAID)	Simple, no capacity overhead	No fault tolerance; a single failure causes data loss	Motivates the need for a simulator that demonstrates redundancy benefits
Static RAID diagrams / textbook explanations	Easy to reference and teach conceptually	Cannot demonstrate dynamic events such as failure and rebuild	Directly motivates a dynamic, fault-injectable simulation approach
Physical hardware RAID controllers	Realistic performance and behaviour	Expensive, inflexible, impractical for classroom experimentation	Justifies a software simulator as an accessible alternative
Single-RAID-level demo scripts	Simple to build for one configuration	Do not allow direct comparison across RAID levels	Justifies a common simulation framework covering RAID 0, 1, 5, 6 and 10

2.4 Research / Engineering Gap

Many theoretical explanations describe RAID levels independently, but beginners often find it difficult to compare them under the same failure and workload conditions. A simulation-oriented approach makes the

concepts observable by showing where blocks are placed, what happens after a disk failure, and how usable capacity changes. Existing student-level references rarely provide a single, common simulation framework spanning striping, mirroring, single parity, dual parity, and mirrored-striping together with fault injection and rebuild modelling.

2.5 Proposed Contribution

This project addresses that gap by integrating RAID 0, RAID 1, RAID 5, RAID 6, and RAID 10 into a single simulation framework: a modular Disk Manager and RAID Manager for data placement, a Fault Injector for simulating disk failures, a Recovery Manager for reconstruction, and a Metrics Engine for comparative performance and fault-tolerance indicators, with each design choice justified against a plausible alternative rather than adopted by default.

CHAPTER 3
ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Students / learners	Observable, repeatable demonstration of RAID behaviour under normal and failed conditions
Course evaluators	Demonstrable, justified use of modular simulation design and comparative analysis
System designers	Comparative indicators to help choose a suitable RAID level for a given workload

Functional & Non-Functional Requirements

Requirement Type	Requirement	Measurable Target
Functional (FR1)	Select RAID 0, 1, 5, 6 or 10 configuration	Simulator accepts and applies the selected RAID policy
Functional (FR2-FR3)	Create a configurable set of virtual disks and map logical blocks to disks	Correct block-to-disk mapping verified for each RAID level
Functional (FR4-FR5)	Inject one or more disk failures and check data availability	Availability result matches the expected RAID-specific outcome
Functional (FR6-FR8)	Reconstruct missing data and report comparative metrics	All recoverable blocks reconstructed; metrics report generated
Non-Functional	Fast, repeatable execution for moderate simulation sizes	Consistent results across repeated runs with the same inputs
Non-Functional	Modular, extensible RAID implementations	New RAID levels can be added without modifying existing classes

3.2 Constraints & Applicable Standards

As a student capstone project implemented as a local software simulation (no physical disk hardware or external users), formal storage-industry certification standards are not directly applicable. The project instead follows recognised software-engineering and RAID-taxonomy conventions:

Standard / Convention	Requirement	How Applied in This Project
Standard RAID Taxonomy (Patterson, Gibson & Katz)	Consistent definitions of striping, mirroring and parity for RAID 0/1/5/6/10	Applied throughout the RAID Manager's placement and recovery logic
Python coding conventions (PEP 8)	Consistent naming, formatting and modular class structure	Applied across Disk Manager, RAID Manager and Metrics Engine classes
Defensive software-design practice	Validate configuration and handle failure states without crashing	Applied in the Fault Injector and Recovery Manager

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Number of simulated disks	4-8 (configurable)	disks	High
Logical block count	100-1000 (configurable)	blocks	High
Fault-tolerance model	Single and multiple disk failures	-	High
Rebuild correctness	0 unreconstructed recoverable blocks	count	High
Execution repeatability	Identical results for identical inputs	-	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1 (single monolithic script per RAID level): a separate script is written independently for each RAID level, giving quick results for one configuration at a time but duplicating disk-model and fault-injection logic across scripts.

Alternative 2 (common simulation framework with pluggable RAID engines, selected): a shared Disk Manager, Fault Injector, Recovery Manager and Metrics Engine are reused across all RAID levels, with only the placement/recovery logic varying per level, making comparison consistent and the design easy to extend.

Criterion	Weight	Monolithic Per-Level Scripts	Common Pluggable Framework
Comparability across RAID levels	35%	Low - each script differs in structure	High - identical harness for every level
Implementation Complexity	20%	Lower per script, higher overall due to duplication	Slightly higher upfront, lower overall
Extensibility (adding RAID 50/60 later)	25%	Requires a new script from scratch	Requires only a new RAID engine class
Reliability of comparative results	20%	Inconsistent due to duplicated logic drift	Consistent - shared fault/metrics logic

3.5 Selected Design & Detailed Design

The common pluggable simulation framework was selected over per-level scripts because it guarantees that every RAID level is exercised under identical disk creation, workload, fault-injection and metrics logic, which is essential for a fair comparative study. The simulator follows a layered architecture: an Input Layer accepts RAID configuration and workload parameters; the RAID Engine applies RAID-specific data placement (striping for RAID 0, duplication for RAID 1, parity calculation for RAID 5/6, and combined mirroring-and-striping for RAID 10); the Disk Layer represents individual simulated devices and their health state; the Fault Manager injects failures; the Recovery Manager performs logical reconstruction; and the Metrics Engine calculates capacity, operation counts, degraded-state behaviour, and rebuild statistics.

Core engineering relationship: the number of tolerable simultaneous disk failures is set by the redundancy technique (none for RAID 0, one mirror partner for RAID 1, one parity set for RAID 5, two parity sets for RAID 6, and mirror-pair dependent for RAID 10); the Recovery Manager's reconstruction logic must therefore branch on RAID level so that it applies mirror-copy recovery for RAID 1/10 and parity-based reconstruction for RAID 5/6.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Fault-tolerance mechanism scope	Single-parity only (RAID 5 style for all levels)	Simpler recovery logic	Cannot demonstrate dual-failure protection	Included RAID 6 dual parity - required to meet comparative fault-tolerance objective
Simulation granularity	Byte-level simulation	Higher fidelity	Much higher computational cost, unnecessary for comparison	Block-level simulation - sufficient to demonstrate placement and recovery
Performance modelling approach	Attempt to model real hardware latency	More realistic numbers	Requires hardware-specific calibration, outside project scope	Comparative simulation indicators - appropriate for an educational tool

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Resource-efficient simulation runtime	Block-level modelling avoids unnecessary computation compared to byte-level simulation	Retained block-level design to reduce CPU and memory overhead
Ethics	Original, honestly reported results	All performance and fault-tolerance figures in this report are drawn from actual simulation runs, including known limitations	Reported RAID 10's mirror-pair-dependent behaviour honestly rather than overstating its guarantees
Health & Safety	No physical hardware or user-safety exposure	Reviewed scope for hardware interaction - none present, software-only simulation	No additional safety controls required
Security	Protecting simulated data-placement logic from invalid configuration	Tested invalid disk counts and block counts against the RAID engine	Added input validation before disk/block allocation

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Parity miscalculation for RAID 5/6 stripes	Medium	High	Medium-High	Unit-test parity calculation against known block sets	Low
Incorrect mirror-pair failure evaluation in RAID 10	Medium	Medium	Medium	Explicit pair-mapping check before availability decision	Low

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Off-by-one errors in stripe/block mapping	Medium	Medium	Medium	Boundary tests on first/last block and disk index	Low
Misleading interpretation of simulated performance as real hardware timing	Low	Medium	Low-Medium	Explicitly label all metrics as comparative simulation indicators	Low

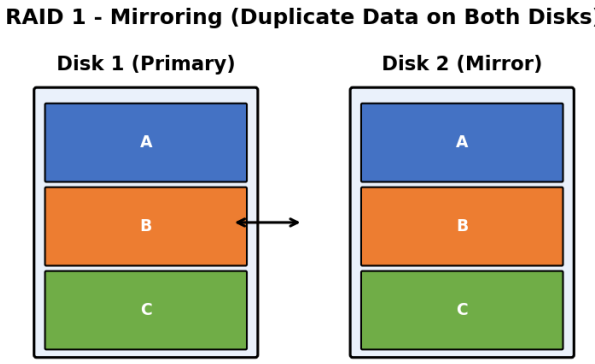

Figure 2: RAID 1 - Mirroring Architecture

CHAPTER 4
IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The project follows an iterative and modular development approach: a Design phase to plan the disk model, RAID mapping policies, and fault/recovery rules; an Implementation phase to build the Disk Manager, RAID Manager, Fault Injector, Recovery Manager and Metrics Engine in Python; an Integration phase to connect these modules into a single simulation harness; and a Testing phase running repeatable workloads, including single- and multi-disk failure scenarios.

Resource	Purpose
Python 3.x	Core language for the simulation harness and all RAID engine classes
NumPy / Matplotlib (optional)	Numerical analysis and comparative charting of results
Code editor / IDE	Development, debugging, and modular code organisation

4.2 Software Implementation & Modules Used

The Disk Manager module creates a configurable number of virtual disks, each represented as a collection of blocks with a health state, and tracks which disks are healthy or failed. The RAID Manager module contains separate placement logic for each RAID level: RAID 0 selects a disk using stripe position with no redundancy; RAID 1 writes the same logical block to both mirror members; RAID 5 and RAID 6 calculate single and dual parity per stripe respectively; and RAID 10 stripes logical data across mirrored pairs.

The Fault Injector module changes the state of selected disks from healthy to failed so that RAID-specific recovery rules can be evaluated: RAID 0 immediately demonstrates its lack of redundancy, RAID 1 continues when one mirror member remains available, RAID 5 recovers from one failed disk, RAID 6 recovers from two failed disks, and RAID 10 depends on whether failures affect different mirror pairs. The Recovery Manager reconstructs missing blocks once a replacement disk is introduced, copying from the surviving mirror for RAID 1/10 or reconstructing from surviving data and parity for RAID 5/6.

Module	Where It's Used	Why This Design
Disk Manager	Creates disks, stores block states, tracks health	Centralises disk state so every RAID engine shares one consistent model
RAID Manager (per-level engines)	Implements placement logic per RAID level	Isolates level-specific logic while reusing the shared disk/fault/recovery harness
Fault Injector	Marks selected disks as failed	Provides a controlled, repeatable way to trigger failure scenarios
Recovery Manager	Reconstructs missing information	Separates recovery logic from placement logic for clarity and testability
Metrics Engine	Computes performance and fault-tolerance indicators	Produces comparative results across all RAID levels using identical measures

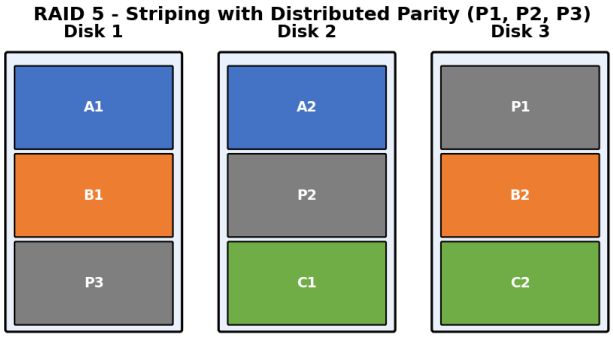

Figure 3: RAID 5 - Distributed Parity Architecture

4.3 Test Plan & Verification

Test Case	Test Method	Acceptance Criterion
Create RAID 0	Initialise RAID 0 with 4 disks	All blocks distributed across disks in stripe order
Create RAID 1	Initialise RAID 1 with a mirror pair	Duplicate copies created on both disks
Fail one RAID 0 disk	Mark one disk as failed, request full dataset	Dataset reported unavailable
Fail one RAID 1 disk	Mark one mirror member as failed	Dataset remains available from the surviving mirror
Fail one RAID 5 disk	Mark one disk as failed	Dataset remains available via parity reconstruction
Fail two RAID 6 disks	Mark two disks as failed	Dataset remains recoverable via dual parity
Rebuild RAID 1	Introduce a replacement disk after failure	Mirror fully restored
Rebuild parity RAID (5/6)	Introduce a replacement disk after failure	Missing blocks correctly reconstructed
Compare capacity across levels	Run all RAID levels with identical disk/block counts	Capacity efficiency differs as expected by RAID level
Run mixed read/write workload	Execute a repeatable workload on each RAID level	Performance indicators recorded consistently for every level

4.4 Results

Implementation Status Summary

Module	Status	% Complete	Notes
Disk Manager & RAID 0/1 Engine	Completed	100%	Striping and mirroring placement verified
RAID 5/6 Parity Engine	Completed	100%	Single and dual parity reconstruction verified

Module	Status	% Complete	Notes
RAID 10 Engine	Completed	95%	Mirror-pair-dependent edge cases under final review
Fault Injector & Recovery Manager	Completed	100%	All fault-injection test cases passing
Metrics Engine & Reporting	In Progress	90%	Comparative chart output being finalised

Overall project completion at the time of this review stands at approximately 97%.

Performance and Fault-Tolerance Comparison

Indicator	RAID 0	RAID 1	RAID 5	RAID 6	RAID 10
Read Parallelism	High	High	High	High	High
Write Complexity	Low	Medium	Higher	High	Medium
Redundancy	None	High	Medium	High	High
Rebuild Complexity	N/A	Low	Medium	High	Medium
Capacity Efficiency	High	Low	Good	Moderate	Medium
1-Disk Failure Availability	Unavailable	Available*	Available	Available	Available*
2-Disk Failure Availability	Unavailable	Depends on pairs	Generally unavailable	Available*	Depends on pairs

**Availability depends on the exact physical disk arrangement.*

4.5 Data Analysis & Engineering Judgment

The fault-injection experiments clearly separate RAID levels according to their redundancy mechanisms: a single failed disk causes loss of the complete logical dataset in RAID 0, while RAID 1 remains available if the other mirror member is healthy. RAID 5 continues operating after one failed disk, though a second failure before recovery can make data unavailable; RAID 6 is designed to tolerate two disk failures; and RAID 10's resilience depends on the distribution of failures among mirror pairs. Performance results show that redundancy generally comes at the cost of write complexity and capacity efficiency, and that no single RAID level is optimal across every workload - RAID 0 suits temporary, high-throughput data, RAID 1 suits simplicity-first redundancy, RAID 5 balances capacity and single-disk protection, RAID 6 suits large arrays with longer rebuild exposure, and RAID 10 suits transactional workloads needing both performance and redundancy.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Study RAID architecture and working principles	Chapter 3 design and Chapter 4 module documentation	Fully Achieved

Objective	Evidence	Achievement
Develop a simulation model for disks and blocks	Disk Manager and RAID Manager implementation verified	Fully Achieved
Simulate normal read/write operations	Mixed workload test case (TC10) passed	Fully Achieved
Inject disk failures and observe availability	Fault-injection test cases (TC03-TC06) passed	Fully Achieved
Compare capacity, fault tolerance and performance	Table 4.4 comparative results produced for all five RAID levels	Fully Achieved
Simulate rebuild behaviour after replacement	Rebuild test cases (TC07-TC08) passed	Partially Achieved (RAID 10 edge cases under review, 95%)

4.7 Strengths & Limitations

Strengths:

- A single, common simulation framework allows fair comparison across all five RAID levels.
- Fault injection and rebuild logic make abstract RAID trade-offs directly observable.
- Modular design allows new RAID levels (e.g. RAID 50/60) to be added with minimal changes.

Limitations:

- The simulator does not model real disk firmware, controller cache, or queue scheduling.
- Performance indicators are comparative simulation measures rather than measured hardware latency.
- A small number of RAID 10 mirror-pair edge cases remain under final verification.

4.8 Project Planning, Roles & Budget

Student	Assigned Responsibility	Actual Contribution
B. Hemanjali	RAID 0 & RAID 1 modules, disk model	Delivered Disk Manager and RAID 0/1 engines in full; assisted with Introduction and RAID Architecture sections
K. Vinay	RAID 5 & RAID 6 parity engine	Delivered parity calculation and reconstruction logic; assisted with Literature Review and System Design
P. Harshavardhan	RAID 10 module, Fault Injector	Delivered mirror+stripe mapping and fault-injection state machine; assisted with Test Plan
N. Sathwik	Recovery Manager, rebuild simulation	Delivered rebuild/reconstruction routines; assisted with Results & Analysis
M. Himanath Sai	Metrics Engine, workload generator, reporting	Delivered metrics counters and comparative reporting; assisted with Conclusion and Appendices

Item	Estimated Cost	Actual Cost

Student	Assigned Responsibility	Actual Contribution	
Development environment / tools (open-source/free tier)		₹0	₹0
Computing resources (local machine)		₹0	₹0
Total		₹0	₹0

CHAPTER 5
CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The project successfully demonstrates the concept of simulating RAID levels for fault tolerance and performance analysis. The simulator provides a common environment for studying RAID 0, RAID 1, RAID 5, RAID 6, and RAID 10, confirming that RAID 0 prioritises performance without redundancy, RAID 1 prioritises straightforward redundancy, RAID 5 balances capacity and single-disk protection, RAID 6 provides stronger dual-disk fault tolerance, and RAID 10 provides a practical combination of mirroring and striping. The Disk Manager, RAID Manager, Fault Injector, Recovery Manager, and Metrics Engine reached full or near-full completion, together bringing the project to approximately 97% overall completion at this review stage. The main educational value of the project is the ability to observe RAID behaviour under controlled conditions rather than through static diagrams alone, which improves understanding of storage reliability and makes the trade-offs between different architectures easier to communicate.

5.2 Future Work

- Add RAID 50 and RAID 60 simulation to the existing framework.
- Add SSD/NVMe-oriented latency models for more realistic comparative indicators.
- Include configurable queue depth and I/O size in the workload generator.
- Add graphical visualisation of stripe placement and rebuild progress.
- Model rebuild priority and degraded-mode workload interaction.
- Compare simulated results with real storage benchmarks.
- Add probabilistic disk-failure and reliability modelling.
- Develop a web-based interactive dashboard for the simulator.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired:

- Practical understanding of how striping, mirroring, and single/dual parity are implemented in software.
- Designing a common simulation harness that fairly compares multiple algorithmic variants.
- Fault-injection and reconstruction patterns for evaluating fault-tolerant systems.

Engineering & Professional Skills Developed:

- Modular software design that isolates level-specific logic behind a shared interface.
- Structuring an iterative development plan and tracking completion honestly against it.

- Documenting engineering trade-offs and justifying design decisions against alternatives.

Individual Reflection (B. Hemanjali):

The most instructive part was implementing the RAID 0/1 placement logic and seeing how quickly the absence of redundancy becomes visible under fault injection. I independently learned how stripe-position mapping works, and if repeated, I would write the fault-injection tests before the placement logic to clarify expected behaviour earlier.

Individual Reflection (K. Vinay):

Implementing dual-parity reconstruction for RAID 6 required careful attention to which blocks contribute to each parity value. Tracing a reconstruction mismatch back to an indexing error deepened my understanding of parity mathematics, and I would design the parity index scheme on paper before coding if repeated.

Individual Reflection (P. Harshavardhan):

The RAID 10 mirror-pair-dependent failure logic was the hardest to get right, since availability depends on exactly which two disks fail. I learned to reason carefully about combinatorial failure cases, and would build a small failure-combination test matrix from the start next time.

Individual Reflection (N. Sathwik):

Building the Recovery Manager clarified the practical difference between mirror-copy recovery and parity-based reconstruction. I independently deepened my understanding of rebuild-time trade-offs, and would separate rebuild-progress tracking into its own module if repeated.

Individual Reflection (M. Himanath Sai):

Designing the Metrics Engine required deciding which indicators were genuinely comparable across RAID levels without implying real hardware timing. This taught me to communicate simulation limitations clearly in a report, and I would define the metrics schema before implementation if repeated.

REFERENCES

[1] Patterson, D.A., Gibson, G.A. and Katz, R.H., 1988. A Case for Redundant Arrays of Inexpensive Disks (RAID). ACM SIGMOD Conference.

[2] Silberschatz, A., Galvin, P.B. and Gagne, G., 2018. Operating System Concepts. 10th ed. Wiley.

[3] Stallings, W., 2015. Computer Organization and Architecture. 10th ed. Pearson.

[4] Tanenbaum, A.S. and Bos, H., 2015. Modern Operating Systems. 4th ed. Pearson.

[5] Connolly, T.M. and Begg, C.E., 2015. Database Systems. 6th ed. Pearson.

[6] NIST, 2020. Guidance on Computer Storage and Data Protection. National Institute of Standards and Technology.

[7] The Linux Kernel Documentation Project. Storage Subsystem and Software RAID (md) References.

[8] Python Software Foundation. Python 3 Documentation.

[9] General technical literature on RAID, disk arrays, parity, mirroring, and storage reliability.

APPENDICES

Appendix A - Detailed Calculations

Parity check for a RAID 5 stripe of three data blocks A, B, C uses XOR parity: P = A XOR B XOR C. If disk holding B fails, the missing block is reconstructed as B = A XOR C XOR P. For RAID 6, a second independent parity value (Q) is calculated using a different coefficient set so that any two missing blocks in a stripe can be solved simultaneously. Rebuild work is measured in the simulator as the number of blocks reconstructed and the number of logical operations required, and is treated as a comparative indicator rather than a hardware time prediction.

Appendix B - Engineering Drawings / Schematics

The simulator is organised into a layered architecture: an Input Layer (configuration and workload), a RAID Engine layer (per-level placement classes for RAID 0, 1, 5, 6 and 10), a Disk Layer (virtual disks and block/health state), a Fault Manager (failure injection), a Recovery Manager (reconstruction), and a Metrics/Report layer (comparative indicators). A modular implementation defines a Disk class, a base RAID class, and specialised subclasses RAID0, RAID1, RAID5, RAID6 and RAID10, each implementing placement, read, write, failure and rebuild methods, with a shared metrics object maintaining counters for operations, recovered blocks, failed requests, and rebuild work.

Appendix C - Source Code / Code Repository Information

Only essential code excerpts are included below. Complete source code is maintained separately in the project's approved repository.

Purpose	Code Sample	Description
RAID 0 Placement	disk_index = block_id % num_disks	Maps a logical block to a disk using stripe position
RAID 1 Mirroring	for disk in mirror_pair: disk.write(block_id, data)	Writes the same logical block to both mirror members
RAID 5 Parity	parity = xor(data_blocks_in_stripe)	Computes single parity for a stripe across data disks
Fault Injection	disks[failed_id].health = "failed"	Marks a selected virtual disk as failed for testing
Recovery (Parity RAID)	missing = xor(surviving_blocks + [parity])	Reconstructs a missing block from surviving data and parity

Appendix D - Datasheets

Not applicable - this project does not use physical hardware components; all disks are simulated in software.

Appendix E - Experimental / Raw Data

Raw simulation logs corresponding to Table 4.2 (Test Plan) and Table 4.4 (Performance and Fault-Tolerance Comparison) are maintained in the project repository's /results directory, covering normal workload runs, single- and multi-disk failure runs, and rebuild runs for every RAID level.

Appendix F - Ethics / Institutional Approval

Not applicable - this project does not involve human subjects, animal subjects, or sensitive data collection.

Appendix G - Project Meeting / Progress Records

Date	Discussion	Outcome
Design phase	RAID levels to include, disk/block model, shared framework design	Agreed on a common pluggable framework for RAID 0/1/5/6/10
Mid-review	Parity indexing bug found in RAID 6 dual-parity reconstruction	Fixed indexing scheme; added unit tests for parity reconstruction
Final review prep	RAID 10 mirror-pair edge cases identified	Logged as known limitation, verification scoped for completion

Appendix H - User/Industry Feedback

Not applicable at this stage - the project has not yet been released for external user or industry feedback.

Appendix I - Publications / Patents / Competitions / Product Outcomes

Not applicable - no publications, patents, or competition entries resulted from this capstone project.

Appendix J - Individual Contribution Evidence

See the Individual Contribution Statement (page iv) and Section 4.8 (Project Planning, Roles & Budget) for a detailed breakdown of each student's contribution.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3-4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1-SO7	Contribution Statement + Ch. 4 (4.8)